EE5203 Lab 8 Guide

ADC: read a potentiometer and a light sensor, then set the LD2 brightness - Basri Erdogan

Mission: Read a potentiometer on A0 with `ADC1` and show the reading in millivolts in Live Watch. Then let the potentiometer set the brightness of LD2 through the TIM2 PWM of Lab 7, with no `HAL_Delay()` anywhere. Then add a light sensor on A1 and turn LD2 into a night-light.

Prepare before class

- ☐ Bring your board, a USB **data** cable, and from your kit: the breadboard, a **10 kΩ potentiometer**, an **LDR**, a **10 kΩ resistor** and jumper wires.
- ☐ Do the **Lab 8 pre-lab quiz** on CATS (two attempts; the higher score counts).
- ☐ Re-read the theory slides up to "Two ADC channels share one converter."
- ☐ Compute: `raw` = 1241 on a 12-bit ADC at 3.3 V. How many millivolts? Which `duty`?
- ☐ Know which register and which bits show that PA0 is an analog pin.

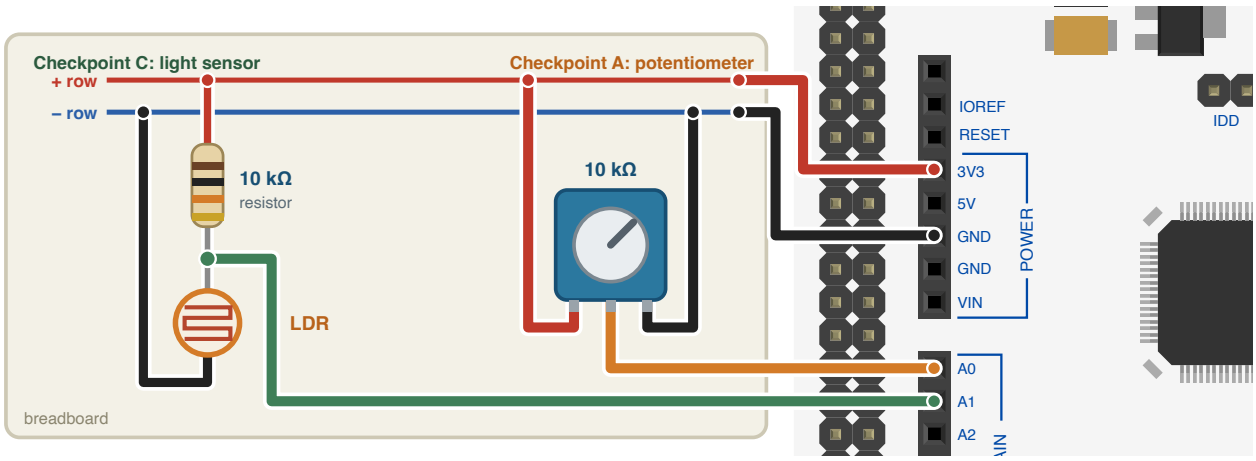
Learning objectives

By checkoff time, you can:

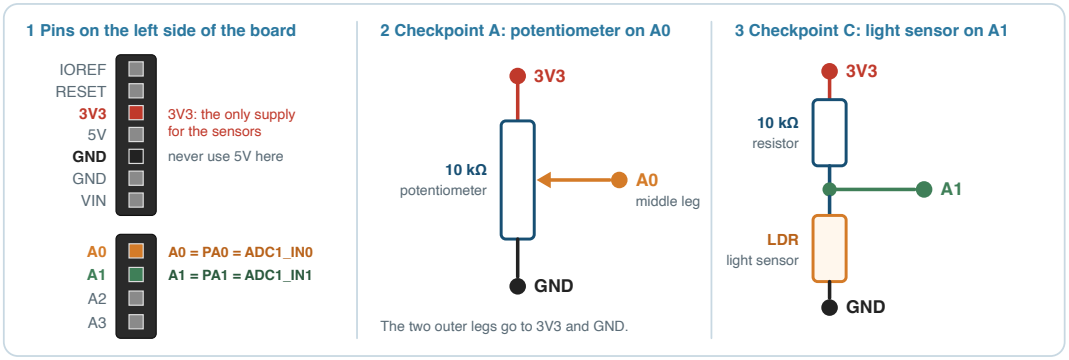
1. configure `ADC1` for two analog pins: 12 bits, single conversion, 84-cycle sampling;
2. read a channel with start, wait, read, on the 10 ms tick;
3. convert an ADC value to millivolts and to a PWM duty with integer arithmetic;
4. read two sensors with one converter and show the channel in the SFR view.

Hardware and files

Item	Value
Board	Nucleo-F401RE: LD2 is on PA5, which is also `TIM2_CH1`
Clock	**HSI, PLL off** (CubeMX step 2): 16 MHz; ADC clock = 16 MHz ÷ 2 = **8 MHz**
Timer	TIM2: 1 ms tick **and** PWM on channel 1, `PSC` 15, `ARR` 999, as in Lab 7
Potentiometer	10 kΩ: outer legs to **3V3** and **GND**, middle leg to **A0** (PA0, `ADC1_IN0`)
Light sensor	from Checkpoint C on: 10 kΩ resistor from **3V3** to **A1**, LDR from **A1** to **GND** (PA1, `ADC1_IN1`)
ADC	`ADC1`: 12 bits, single conversion, sampling time **84 cycles** (12 µs for one reading)
Project	new CubeMX project `EE5203_Lo8_<studentnumber>`, built in VS Code
File you edit	`main.c`, only inside the `USER CODE` regions



The same two circuits as diagrams:



One jumper takes **3V3** to the + row of the breadboard and one takes **GND** to the − row. An analog pin must stay between

0 V and 3.3 V: never connect the **5V** pin to the breadboard in this lab.

Configure in CubeMX

Start of the session: if `C:\Work\<studentnumber>` does not contain your Lab 7 project, copy **all** your project folders from `H:\EE_Labs` into it first.

1. **STM32CubeMX:** Board Selector → **NUCLEO-F401RE** → accept the default initialization.
2. **Clock Configuration tab: System Clock Mux** = **HSI**; **AHB**, **APB1** and **APB2** prescalers = **/1**. Every box on the right reads **16**, as in Lab 7.
3. **PA5:** click the pin and choose `TIM2_CH1`, as in Lab 7. Do this before step 4.
4. **Timers → TIM2:** Clock Source = `Internal Clock`; Channel1 = `PWM Generation CH1`. Parameter Settings: Prescaler `15`, Counter Period `999`; PWM Generation Channel 1: Mode `PWM mode 1`, Pulse `0`.
5. **PA0 and PA1:** click **PA0** and choose `ADC1_IN0`; click **PA1** and choose `ADC1_IN1`. Both pins turn green.
6. **Analog → ADC1 → Parameter Settings:** Resolution `12 bits (15 ADC Clock cycles)` and Continuous Conversion Mode `Disabled` are the defaults: check them. Open **ADC_Regular_ConversionMode → Rank 1**: Channel = `Channel 0`, Sampling Time = `84 Cycles`. The default is `3 Cycles`: change it.
7. **System Core → NVIC**: enable **TIM2 global interrupt**. The ADC needs no interrupt.
8. Project Manager: name `EE5203_L08_<studentnumber>`, location `C:\Work\<studentnumber>`, Toolchain/IDE = **CMake** → Generate Code.
9. **Copy `C:\Users\Public\EE5203_template\.vscode` into the project folder**, as in every lab. (Own laptop: use **Set Up STM32Cube project(s)**, as in Lab 5.)
10. **Verify, do not add:** in `main.c`, `MX_ADC1_Init()` with `ADC_RESOLUTION_12B`, `ADC_CHANNEL_0` and `ADC_SAMPLETIME_84CYCLES` and `MX_TIM2_Init()` with `TIM_OCMODE_PWM1`; in `stm32f4xx_hal_msp.c`, `HAL_ADC_MspInit()` with `GPIO_PIN_0|GPIO_PIN_1` and `GPIO_MODE_ANALOG`, and `GPIO_AF1_TIM2` (PA5); in `stm32f4xx_it.c`, `TIM2_IRQHandler()`.

Checkpoint A - The potentiometer in millivolts

Put the potentiometer on the breadboard and wire it as in the figure: outer legs to the + row (3V3) and the − row (GND), middle leg to A0. Fill the **Predicted** columns of the table below before you build. Three additions:

1. **In `USER CODE BEGIN PV`**, the millisecond counter of Lab 7 and two results. They are global variables, so Live Watch can show them:

```
volatile uint32_t g_ms = 0;
uint32_t raw_pot = 0, mv_pot = 0;     /* potentiometer: ADC value and millivolts */
```

2. **In `USER CODE BEGIN 4`**, the tick callback, as in Lab 7:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    g_ms++;                                /* TIM2 update: every 1 ms */
}
```

3. **In `USER CODE BEGIN 2`**, start the tick; **in `USER CODE BEGIN 3`**, one reading every 10 ms: start, wait, read, then convert:

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);                 /* 1 ms tick: g_ms */
uint32_t t_adc = 0;

/* USER CODE BEGIN 3 */
if ((g_ms - t_adc) >= 10U) {                   /* every 10 ms */
    t_adc = g_ms;
    HAL_ADC_Start(&hadc1);                     /* start one conversion      */
    HAL_ADC_PollForConversion(&hadc1, 2);      /* wait for EOC, about 12 us */
    raw_pot = HAL_ADC_GetValue(&hadc1);        /* read ADC1->DR             */
    mv_pot  = (raw_pot * 3300u) / 4095u;       /* multiply first            */
}
```

4. Build (**Ctrl+Shift+B**). In `.vscode/launch.json`, below `"runToEntryPoint": "main",` paste the line `"liveWatch": { "enabled": true, "samplesPerSecond": 4 },` as in Lab 7. Start the debugger (**F5**), Continue. In **Run and Debug**, open **Cortex Live Watch** (own laptop: **STM32Cube Live Watch**), click +, add `raw_pot` and `mv_pot`.

Potentiometer position	Predicted `raw_pot`	Predicted `mv_pot`	Measured `raw_pot`	Measured `mv_pot`
fully toward GND				
about the middle				
fully toward 3V3				

Evidence for Checkpoint A: `raw_pot` follows the potentiometer from about 0 to about 4095, and `mv_pot` from about 0 to about 3300. Pause: in the SFR view `ADC1->DR` holds the same number as `raw_pot`; `GPIOA->MODER` bits 1:0 read `11`

(PA0 is analog); SMP0 in ADC1->SMPR2 reads 100 (84 cycles). Copy the table into USER CODE BEGIN Header as a comment. **Screenshot A_live.png:** Live Watch with raw_pot and mv_pot.

Checkpoint B - The potentiometer sets the LD2 brightness

The reading becomes a PWM duty: 0 … 4095 is scaled to 0 … 1000. The top value is ARR + 1: LD2 fully on, the +1 rule of Lab 7. Three additions:

1. **In USER CODE BEGIN PV**, below the potentiometer variables:

```
uint32_t duty = 0;                    /* 0 ... 1000, written to TIM2->CCR1 */
```

2. **In USER CODE BEGIN 2**, below the tick line, start the PWM:

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);     /* PWM on PA5 (LD2) */
```

3. **In USER CODE BEGIN 3**, inside the 10 ms block, below the mv_pot line:

```
duty = (raw_pot * 1000u) / 4095u;             /* 0 ... 1000 */
TIM2->CCR1 = duty;
```

Evidence for Checkpoint B: the potentiometer moves LD2 from dark to fully on with no steps. Add duty to Live Watch: it goes from 0 to 1000. Pause: TIM2->CCR1 in the SFR view holds the same number as duty. No HAL_Delay() in main.c. **Predict, then check:** the potentiometer is at the middle. What is duty? Then turn the potentiometer slowly from one end to the other. In which half of the turn does the brightness change more? Write one sentence: why? (Lab 7, the level table.) **Screenshot B_duty.png:** Live Watch with raw_pot, mv_pot and duty, and your USER CODE BEGIN 3 lines.

Checkpoint C - A light sensor on a second channel

Add the light sensor as in the figure: the 10 kΩ resistor from the + row (3V3) to A1, the LDR from A1 to the − row (GND). The potentiometer stays on A0. ADC1 converts one channel at a time, so a function selects the channel before each conversion. Three changes:

1. **In USER CODE BEGIN 0**, the function:

```
uint32_t adc_read(uint32_t channel)
{
    ADC_ChannelConfTypeDef s = {0};
    s.Channel      = channel;
    s.Rank         = 1;
    s.SamplingTime = ADC_SAMPLETIME_84CYCLES;
    HAL_ADC_ConfigChannel(&hadc1, &s);            /* select the channel */
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 2);
    return HAL_ADC_GetValue(&hadc1);
}
```

2. **In USER CODE BEGIN PV**, below duty:

```
uint32_t raw_ldr = 0;                 /* light sensor: ADC value */
```

3. **In USER CODE BEGIN 3**, the 10 ms block now reads both sensors, and the light sensor sets the duty:

```
if ((g_ms - t_adc) >= 10U) {                  /* every 10 ms */
    t_adc = g_ms;
    raw_pot = adc_read(ADC_CHANNEL_0);        /* potentiometer, PA0 */
    raw_ldr = adc_read(ADC_CHANNEL_1);        /* LDR,  PA1 */
    mv_pot  = (raw_pot * 3300u) / 4095u;
    duty    = (raw_ldr * 1000u) / 4095u;      /* dark room, bright LD2 */
    TIM2->CCR1 = duty;
}
```

Evidence for Checkpoint C: add raw_ldr to Live Watch. Cover the LDR with your hand: raw_ldr rises and LD2 gets brighter. Shine your phone light on it: raw_ldr falls and LD2 gets dimmer. The potentiometer still changes raw_pot, but it does not change raw_ldr or LD2. Pause: SQ1 in ADC1->SQR3 reads 1, because channel 1 was converted last. In USER CODE BEGIN Header, write one sentence: why does a dark room give a **larger** raw_ldr? Use the divider. **Screenshot C_night.png:** Live Watch with raw_pot, raw_ldr and duty.

Checkpoint D - The potentiometer sets a servo angle (optional, not graded)

Only if your kit has a servo and time remains. TIM3 makes the 50 Hz servo signal on PA6 (D12), as in the Lab 7 extension. Predict first: raw_pot = 1024 gives which TIM3->CCR1, and which angle?

1. **CubeMX:** click **PA6** and choose TIM3_CH1. **Timers → TIM3:** Clock Source = Internal Clock; Channel1 = PWM Generation CH1; Prescaler 15, Counter Period 19999, Pulse 1500. Generate Code: your USER CODE lines stay.

2. **Wiring:** servo signal (orange) to **D12**, red to **5V**, brown to **GND**. The servo is the only part on the 5V pin: wire it straight to the servo, never to the breadboard rows.
3. **In `USER CODE BEGIN 2`**, start the servo signal; **in `USER CODE BEGIN 3`**, inside the 10 ms block, one more line:

```
HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);          /* USER CODE BEGIN 2 */

TIM3->CCR1 = 500u + (raw_pot * 2000u) / 4095u;     /* 10 ms block */
```

Evidence for Checkpoint D: the potentiometer turns the servo from one end to the other, and `TIM3->CCR1` in the SFR view matches your prediction. If the servo buzzes at an end, it is at its stop (Lab 7): stay a little inside that end.

Individual extension - Fix an AI-written night-light (optional, click)

Optional. Open this only after Checkpoint C is shown and time remains (instead of Checkpoint D, or after it). A classmate asked an AI tool for an on/off night-light. The code compiles, but it does not work on your board. Call `night_light()` from your 10 ms block instead of the `raw_ldr`, `duty` and `CCR1` lines of Checkpoint C.

```
void night_light(void) {
    ADC_ChannelConfTypeDef s = {0};
    s.Channel = ADC_CHANNEL_4;          // LDR input
    s.Rank = 1;
    s.SamplingTime = ADC_SAMPLETIME_3CYCLES;
    HAL_ADC_ConfigChannel(&hadc1, &s);
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 2);
    raw_ldr = HAL_ADC_GetValue(&hadc1);
    if (raw_ldr > 2000u)
        TIM2->CCR1 = 1000;              // on
    else
        TIM2->CCR1 = 0;                 // off
}
```

1. **LD2 ignores your hand.** Pause after a read. Find `SQ1` in `ADC1->SQR3` and compare its channel number with the Hardware table. Fix the code. Screenshot: `E_sqr3.png`.
2. **The light reading follows the potentiometer.** Keep the light steady and turn the potentiometer end to end. Write the lowest and highest `raw_ldr`. Change the sampling time to 84 cycles and repeat.

Sampling time	Lowest `raw_ldr`	Highest `raw_ldr`	Spread
3 cycles			
84 cycles			

3. **Switch once, without flicker.** Your switching point is T = 1500 + 10 × (last two digits of your student number). LD2 turns on above T + h and off below T − h. Choose h larger than your 84-cycle spread. Shade the LDR slowly: LD2 must switch once.

Evidence: `E_sqr3.png`; the table, T and h in `USER CODE BEGIN Header`, with one sentence on why h must be larger than the spread; the slow-shade demo.

Acceptance checklist

- ☐ `.ioc`: HSI 16 MHz; PA0 `ADC1_IN0`, PA1 `ADC1_IN1`; 12 bits, sampling time 84 cycles; PA5 `TIM2_CH1`, PWM mode 1, `PSC` 15 / `ARR` 999; TIM2 enabled in NVIC.
- ☐ Prediction table filled before the first build; `mv_pot` computed with the multiplication first.
- ☐ SFR evidence for Checkpoint A: `ADC1->DR` = `raw_pot`, `MODER` bits 1:0 = `11`.
- ☐ The potentiometer moves LD2 from dark to fully on; `duty` reaches 1000.
- ☐ No `HAL_Delay()` anywhere: every reading is on the 10 ms tick.
- ☐ LD2 gets brighter when the LDR is covered; `SQ1` in `ADC1->SQR3` reads 1.
- ☐ Optional (D): the servo follows the potentiometer; `TIM3->CCR1` matches the prediction.

Individual oral check

- `raw_pot` reads 1241. How many millivolts is that, and what is `duty`?
- Why is the scale factor 1000 and not 999?
- The sampling time is 84 cycles and the ADC clock is 8 MHz. How long is one conversion?
- `raw_pot` and `raw_ldr` are read on the same tick. Are they from the same instant? Why?
- Why does covering the LDR make `raw_ldr` larger? What changes if the LDR and the resistor swap places?

Submit

Submit `main.c` as it is at the end of the session, `A_live.png`, `B_duty.png` and `C_night.png`.

Marks (10): pre-lab quiz on CATS 1 · A potentiometer in millivolts with SFR evidence 2 · B potentiometer sets the LD2 brightness 2 · C two channels and the night-light 3 · oral check (one question) 2. Checkpoint D and the extension are optional and not graded.

If you are stuck

Walk the signal from left to right and fix the **first** silent stage: potentiometer voltage on A0 → PA0 analog (`MODER` bits 1:0 = `11`) → `g_ms` growing → the 10 ms block runs (breakpoint) → `ADC1->DR` changing → `raw_pot` → `duty` → `TIM2->CCR1` → LD2. **`raw_pot` stays at 0 or at 4095?** The middle leg of the potentiometer is not on A0, or an outer leg is not on 3V3 or GND. **`raw_pot` never changes from its first value?** `g_ms` is frozen: `HAL_TIM_Base_Start_IT()` is missing, or TIM2 global interrupt is not enabled in NVIC (step 7). **`mv_pot` is 0 until the very end of the turn?** The division comes before the multiplication. **LD2 stays dark in Checkpoint B?** `HAL_TIM_PWM_Start()` is missing, or PA5 is still `GPIO_Output` in the `.ioc` (step 3). **`raw_ldr` follows the potentiometer?** A1 is not connected, or the sampling time is still 3 cycles (step 6, or the line in `adc_read()`). **LD2 gets dimmer when you cover the LDR?** The LDR and the resistor have swapped places. **LD2 is never fully dark in a bright room?** Correct: `raw_ldr` is not 0 in light. The extension turns this into a clean on/off switch. **Live Watch empty?** Add the `liveWatch` line, then restart the debugger (Shift+F5, F5).